Philosophers

argümanlar→

1. **number_of_philosophers**
2. **time_to_die**
3. **time_to_eat**
4. **time_to_sleep**
5. **[number_of_times_each_philosopher_must_eat](opsiyonel)**

Genel kod akışı:

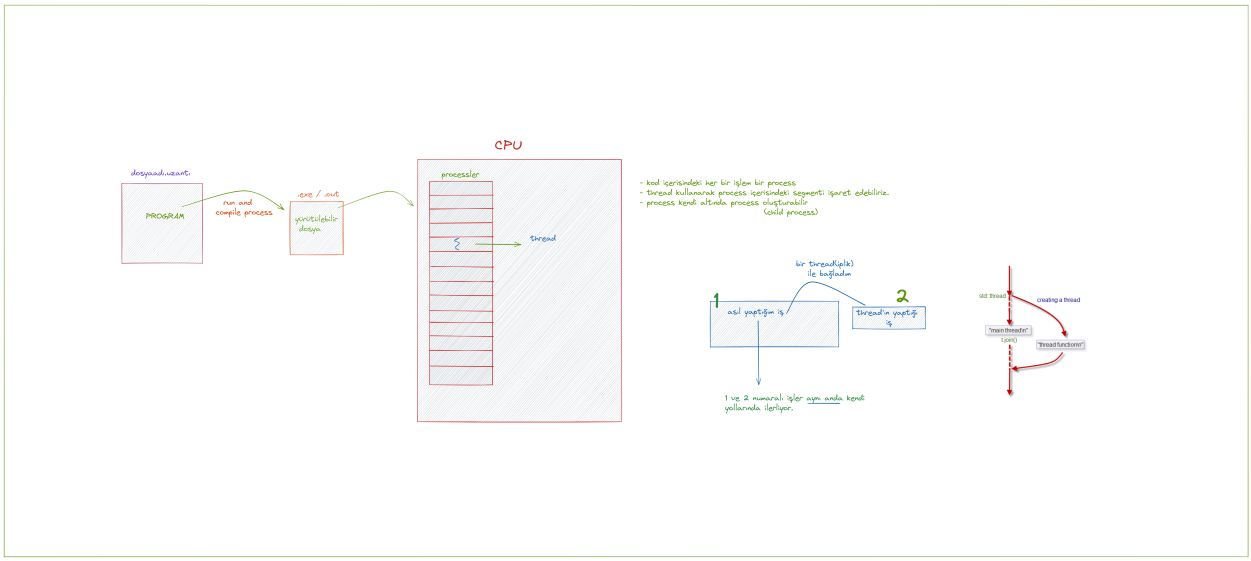
1. argümanlar sayı mı diye ve 5 ya da 6(bir tanesi opsiyonel) argüman var mı diye kontrol edilmesi.
2. philo için t_philo türünde yer açılması
3. çatallar için mutexlerin oluşturulması
4. herkesin çatalının tanıtılması
5. philosopların tanımlanması
6. thread oluşturma
7. main ve diğer threadlerin arasında zamansal eşgüdüm sağlayabilmek adına time_to_die * 1000 yapıyoruz tek filozof varsa
8. dead check
9. freeleme

= thread ve mutex =

- normal bir program tek bir koldan akar ve aynı anda sadece tek bir komut çalıştırır. Programın yönettiği iş sayısı yetmediğinde ya da **aynı anda** birden fazla işin akması

gerektiğinde thread kullanırız. Program birden fazla parçaya (thread) bölünerek işler.

- thread process'in içerisindeki segmentleri işaret eder.



=process thread farkı=

PROCESS	THREAD
-thread process'in belli bir segmentini işaret eder.	
	- thread oluşturmak ve sonlandırmak daha kısa sürer.
- birbirlerinden izole çalışır	- aynı bellek kaynağını paylaşır
- daha fazla kaynak tüketir	- daha az kaynak tüketir
- sırayla çalışırlar. bir process bloklanırsa diğeri çalışamaz. öncelikle ilk process'in işini bitirmesi beklenir.	- görece **bağımsız çalışırlar.** başka bir ****thread bloklansa** bile diğeri çalışmaya devam edebilir.

=**thread blonklanması(Mutexleme)=

- ortak kaynak kullanımından ötürü karışıklıklar ortaya çıkabilir. threadler aynı anda aynı değişken üzerinde farklı değişiklikler yapmak isteyebilirler ya da datarace

oluşabilir. bu durumlarda değişkenleri korumak için **mut**ual **ex**clusion (mutex/karışıklık önleyiciler) kullanılır.

- bir mutex → locked ya da unlocked olabilir.
- **pthread_mutex_init()** mutexin default değerlerle başlangıcını sağlar.
- mutexlerin türü **pthread_mutex_t** olmalıdır (kendisi bir struct yapısı)
- **pthread_mutex_lock() → içerisindeki pthread_mutex_t türündeki değişkeni kilitler.**
- **pthread_mutex_unlock() → içerisindeki pthread_mutex_t türündeki değişkenin kilidini açar.**

```
pthread_mutex_t death;

pthread_mutex_init(philo->death, NULL);
pthread_mutex_lock(philo->death);
pthread_mutex_unlock(philo->death);
```

pthread kütüphenesi

#include <pthread.h>

1. **int pthread_create(pthread_t **thread*, const pthread_attr_t **attr*, void * (**start*)(void *), void **arg*);**

   → attr(attribute) parametresi niteliklerinde yeni bir thread oluşturmak için kullanılır.

   parametreler:

   - ***pthread_t *thread : oluşturulacak thread (philo[1].thread)***
   - ***const pthread_attr_t *attr : thread'in niteliği. NULL ise default tanımlanır.***

- **void *(**start*)(void *) : thread'in main fonksiyonu. Oluşturduğum her thread'i bu fonksiyona gönderir.**
- **void *arg : başlatılacak argüman (philo[i])***

return value:

başarı durumunda **0**,

hata durumunda **EAGAIN, EFAULT, EINVAL**

2. **pthread_mutex_lock( ) ve pthread_mutex_unlock( )**

- içerisindeki pthread_mutex_t türündeki değişkeni kilitler veya açar. data race oluşmasın diye ph→lock'u kilitleyip açıyoruz sürekli.

sys/time ve sys/types kütüphanesi

 # include <sys/time.h>

#include <sys/types.h>

- **<sys/time.h>** has an **timeval structure** like that:

```
struct timeval
{
    <type> time_t;
    <type> tv_sec; //second için
    <type> suseconds_t;
    <type> tv_usec ;   //microseconds  için
    ...

}
```

- **time_t** ve **suseconds_t** **<sys/types.h>** da tanımlı

<sys/time.h> içerisindeki fonksiyonlar:

- **int getitimer(int, struct itimerval *);**
- **intsetitimer(int, const struct itimerval *, struct itimerval *);**
- **intgettimeofday(struct timeval *, void *); (kullanıyoruz)**
- **intselect(int, fd_set *, fd_set *, fd_set *, struct timeval *);**
- **intutimes(const char *, const struct timeval [2]);**

→ **intgettimeofday(struct timeval *tp, void *tzp)**

- şu an ki zamanı alır timeval structında depolar(tp ile point edilmiş)
- tv_sec'de saniyeyi tv_usec'de mikrosaniyeyi

kullanımı:

```
#include <sys/time.h>
#include <stdio.h>

int main() {
  struct timeval current_time;
  gettimeofday(&current_time, NULL);
  printf("seconds : %ld\nmicro seconds : %ld",
    current_time.tv_sec, current_time.tv_usec);

  return 0;
}
```

kodda bu şekilde bir şey yapmamızın sebebi:

- philo->ms = (philo->tv.tv_sec * 1000) + (philo->tv.tv_usec / 1000);
- gettimeofday'i kullandığımızda görüyoruz ki: mikrosaniye çok hızlı değişirken saniye ise yavaş değişiyor (kod satırları arasında yavaş değişiyor normal düşünme).

ikisinin ortasında ortalama bir değişime sahip değişkene ihtiyacımız var bu yüzden saniyeyi 1000’ le çarpıp mikrosaniyeyi 1000’e bölüp toplayıp milisaniye cinsinden karşılığını buluyoruz. böylelikle filozoflardaki değişimi kontrol etmemiz kolaylaşıyor. AYRICA time_to_die değerlerini **kullanıcı bize milisaniye cinsinden veriyor** her türlü çevirmek lazım.

→ 1 saniye = 1000 milisaniye

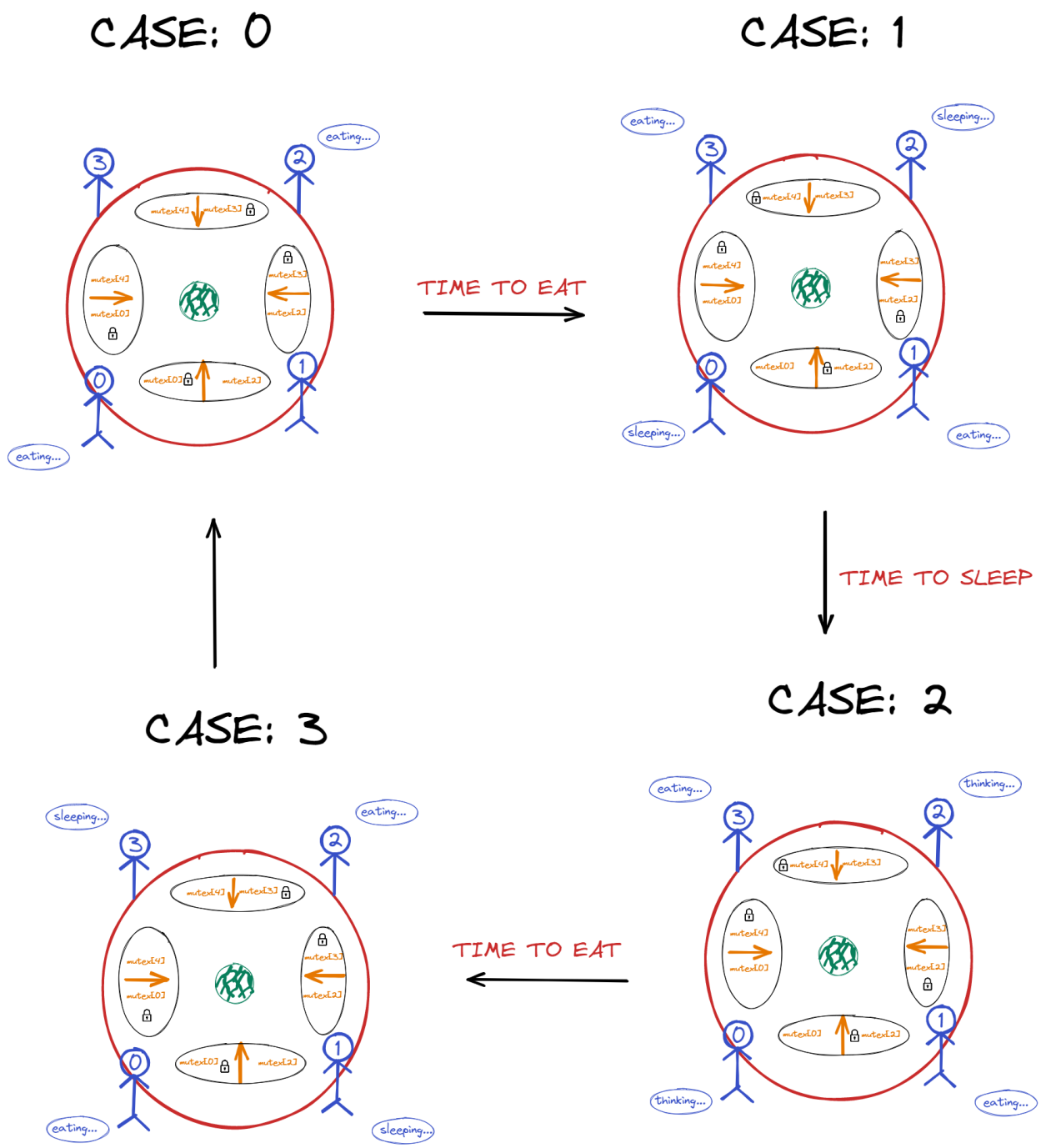
→ 1 milisaniye = 1000 mikrosaniye

data race kontrolü:

- data race: aynı değişkenin başka processler tarafından eş zamanlı değiştirilmeye çalışılması ve karışıklık olması.
- ThreadSanitizer is a data race detector for C/C++. Data races are one of the most common and hardest to debug types of bugs in concurrent systems. A data race occurs when two threads access the same variable concurrently and at least one of the accesses is write

```
NAME    = philo
CC = gcc
INC = philo.h
CFLAGS = -Wall -Wextra -Werror -pthread -fsanitize=thread -g
RM = rm -rf

SRCS    =   main.c \
```



hatalar ve çözümleri:

- **undefined reference to `pthread_mutex_init' ->** gcc <dosya>.c -lpthread ile çalıştırınca düzeliyor.

- **# include <pthread.h> tanınmıyor hatası vscode'da** → mingw installation manager'dan mingw base system kısmından pthread içeren 3 şeyi de tikle. apply changes de

References:

https://devnot.com/2021/thread-nedir-detayli-bir-thread-incelemesi/